Klasifikasi Ensemble untuk Deteksi Fraud pada Dataset dengan Class Imbalance Ekstrem

Laporan Tugas Praktikum Data Mining — Credit Card Fraud Detection (284.807 transaksi)

Nama : **Firdaus Mangkona**	NIM : **5054251003**
Mata Kuliah : Data Mining	Tanggal : 16 Oktober 2026

> **Ringkasan.** Praktikum ini membandingkan tujuh konfigurasi model *ensemble* (*Random Forest* dan *Hist Gradient Boosting*) untuk mendeteksi transaksi kartu kredit fraud pada dataset yang sangat tidak seimbang — hanya **492 dari 284.807 transaksi (0,1727%)** yang merupakan fraud, dengan rasio **±578 : 1**. Tiga pendekatan penanganan kelas tidak seimbang diuji: **SMOTE**, **random undersampling**, dan **pembobotan kelas** (`class_weight="balanced"`); seluruh resampling hanya dikenakan pada data latih.
>
> Hasil terbaik diperoleh **Hist Gradient Boosting + pembobotan kelas** dengan **F1-score 0,8889**, **precision 0,9231**, **recall 0,8571**, **ROC-AUC 0,9828**, dan **PR-AUC 0,8808** — menangkap 84 dari 98 transaksi fraud hanya dengan 7 *false alarm*. Penanganan imbalance menaikkan F1 model ini sebesar **+42,02 poin persen** dibanding baseline-nya. Sebaliknya, *undersampling* menghasilkan recall tertinggi (0,9184) namun tidak dapat dipakai karena precisionnya hanya 0,0424 (2.034 false alarm). *Dummy classifier* yang selalu memprediksi "Normal" memperoleh **accuracy 99,8280%** dengan recall **0**, membuktikan bahwa **accuracy bukan metrik yang layak** untuk kasus ini.

1. Pendahuluan

1.1 Latar Belakang

Deteksi fraud transaksi kartu kredit adalah persoalan klasifikasi biner dengan *class imbalance* ekstrem: transaksi fraud sangat jarang terjadi, tetapi biaya yang ditimbulkan jauh lebih besar daripada biaya pemeriksaan transaksi normal. Kondisi ini membuat metrik *accuracy* praktis tidak berguna — sebuah model yang selalu menjawab "Normal" sudah memperoleh akurasi di atas 99%. Selain itu, algoritma klasifikasi standar dilatih untuk meminimalkan kesalahan total, sehingga kelas minoritas yang jumlahnya sangat sedikit "dikorbankan" demi menekan kesalahan pada kelas mayoritas.

Laporan ini menyusun dan membandingkan model *ensemble* untuk memisahkan transaksi fraud dari transaksi normal, sekaligus mengukur dampak nyata dari tiga pendekatan penanganan *class imbalance* yang umum digunakan.

1.2 Tujuan

1. Mengidentifikasi dan memvisualisasikan distribusi kelas pada dataset transaksi kartu kredit.
2. Menerapkan pembagian data latih dan data uji 80:20 dengan `stratify=y`, serta teknik resampling (SMOTE dan undersampling) yang hanya diterapkan pada data latih.
3. Membangun model klasifikasi berbasis *ensemble* (Random Forest dan Gradient Boosting) dan membandingkan performa **tanpa** dan **dengan** penanganan *class imbalance*.
4. Mengevaluasi model dengan precision, recall, F1-score, ROC-AUC, PR-AUC, dan *confusion matrix* pada kelas minoritas (fraud), serta menjelaskan keterbatasan metrik *accuracy*.

1.3 Dataset

Dataset yang digunakan adalah **Credit Card Fraud Detection** dari ULB (Université Libre de Bruxelles) Machine Learning Group yang dipublikasikan di Kaggle (`kaggle.com/datasets/mlg-ulb/creditcardfraud`), berkas `creditcard.csv` (±144 MB). Dataset berisi transaksi kartu kredit oleh pemegang kartu Eropa pada September 2013.

Tabel 1. Struktur dataset yang digunakan

Aspek	Keterangan
Jumlah baris	284.807 transaksi
Jumlah kolom	31 kolom (30 fitur + 1 target)
Fitur `V1` – `V28`	28 komponen utama hasil PCA (fitur asli dirahasiakan), sudah terskala dan bebas multikolinearitas
Fitur `Time`	Detik sejak transaksi pertama (rentang 0 – 172.792 detik ≈ 2 hari)
Fitur `Amount`	Nominal transaksi (0 – 25.691,16)
Target `Class`	0 = transaksi normal, 1 = transaksi fraud
Kualitas data	Tidak ada *missing value*; terdapat 1.081 baris duplikat

2. Metodologi

2.1 Praproses Data

Seluruh kolom bertipe numerik dan tidak memuat *missing value*, sehingga tidak diperlukan imputasi. Fitur `V1` – `V28` sudah merupakan keluaran PCA yang terskala, sedangkan `Time` dan `Amount` masih berada pada skala asli. Keduanya distandardisasi menggunakan `StandardScaler` yang **di-fit hanya pada data latih** (lalu diterapkan pada data uji) untuk mencegah kebocoran informasi dari data uji.

Standardisasi tersebut bukan sekadar formalitas: SMOTE membangkitkan sampel sintetis berdasarkan **jarak Euclidean antar tetangga terdekat**. Tanpa standardisasi, kolom `Amount` yang bernilai ribuan akan mendominasi perhitungan jarak dibanding `V1` – `V28` yang bernilai satuan, sehingga sampel sintetis yang dihasilkan menjadi tidak realistis.

2.2 Pembagian Data Latih dan Data Uji

Data dibagi dengan perbandingan **80:20** menggunakan `train_test_split(test_size=0.2, random_state=42, stratify=y)`. Parameter `stratify=y` menjaga proporsi kelas tetap sama pada kedua bagian — hasilnya proporsi fraud data latih (0,1729%) dan data uji (0,1720%) berbeda hanya $8{,}8\times10^{-6}$, sangat dekat dengan proporsi keseluruhan (0,1727%).

Pembagian dilakukan sebelum resampling. Data uji tidak boleh memuat sampel sintetis maupun hasil undersampling agar evaluasi menggambarkan performa pada distribusi asli di dunia nyata.

Tabel 2. Komposisi data latih dan data uji

Subset	Normal (0)	Fraud (1)	Total	Proporsi fraud
Data latih (80%)	227.451	394	227.845	0,1729%
Data uji (20%)	56.864	98	56.962	0,1720%
Dataset penuh	284.315	492	284.807	0,1727%

2.3 Pendekatan Penanganan Class Imbalance

Tiga pendekatan diuji dan seluruhnya **hanya diterapkan pada data latih**:

(a) SMOTE — *Synthetic Minority Over-sampling Technique*

SMOTE membangkitkan sampel fraud **sintetis** baru dengan cara melakukan interpolasi linier antara sebuah sampel minoritas dan *k* tetangga terdekatnya di ruang fitur (`k_neighbors=5`, `random_state=42`). Berbeda dari *random oversampling* yang hanya menduplikasi baris (sehingga memicu *overfitting*), SMOTE memperkaya variasi kelas

minoritas. Pada data latih, 394 transaksi fraud diperbanyak menjadi **227.451** sampel sehingga total data latih menjadi **454.902 baris** dengan komposisi 50% : 50%.

(b) Random Undersampling

Pendekatan sebaliknya: transaksi normal dikurangi secara acak hingga setara dengan jumlah transaksi fraud, yaitu 394 baris per kelas (**788 baris latih** — hanya 0,35% dari data latih asli). Pendekatan ini membuat pelatihan sangat cepat, tetapi mengorbankan hampir seluruh informasi kelas mayoritas.

(c) Pembobotan Kelas (*cost-sensitive learning*)

Parameter `class_weight="balanced"` memberi bobot berbanding terbalik dengan frekuensi kelas pada fungsi objektif model, sehingga kesalahan pada kelas minoritas dihukum jauh lebih berat. Keunggulan pendekatan ini: **tidak ada satu baris data pun yang ditambah atau dibuang**, sehingga struktur dan ukuran dataset tetap utuh dan waktu latih tidak bertambah.

Tabel 3. Komposisi data latih setelah setiap perlakuan resampling (data uji tidak diubah)

Data latih	Normal (0)	Fraud (1)	Total baris	Proporsi fraud
Asli (tanpa perlakuan)	227.451	394	227.845	0,17%
+ SMOTE (oversampling)	227.451	227.451	454.902	50,00%
+ Random undersampling	394	394	788	50,00%
Data uji (distribusi asli)	56.864	98	56.962	0,17%

2.4 Model Ensemble dan Konfigurasi Eksperimen

Dua algoritma *ensemble* digunakan: **Random Forest** (200 pohon, bagging atas pohon keputusan) dan **Hist Gradient Boosting** (500 iterasi, *learning rate* 0,05, boosting gradien berbasis histogram). Agar perbandingan benar-benar mengisolasi pengaruh penanganan *class imbalance*, **seluruh hyperparameter dan data latih dijaga identik** dalam satu kelompok model — satu-satunya yang berbeda adalah perlakuan terhadap ketidakseimbangan kelas.

Khusus Hist Gradient Boosting, *early stopping* dimatikan (`early_stopping=False`). Alasannya: validasi internal *early stopping* memakai pembagian acak **tanpa stratifikasi**, sehingga himpunan validasi hanya memuat sekitar 19 transaksi fraud dan penentuan iterasi terbaik menjadi ditentukan oleh derau — hasilnya tidak stabil dan menjadi variabel pengganggu yang menutupi efek sebenarnya dari penanganan imbalance.

Sebagai pembanding ditambahkan `DummyClassifier(strategy="most_frequent")` yang selalu memprediksi kelas mayoritas, untuk mendemonstrasikan keterbatasan metrik *accuracy*.

2.5 Metrik Evaluasi

Evaluasi difokuskan pada performa kelas minoritas (fraud), dengan data uji berdistribusi asli:

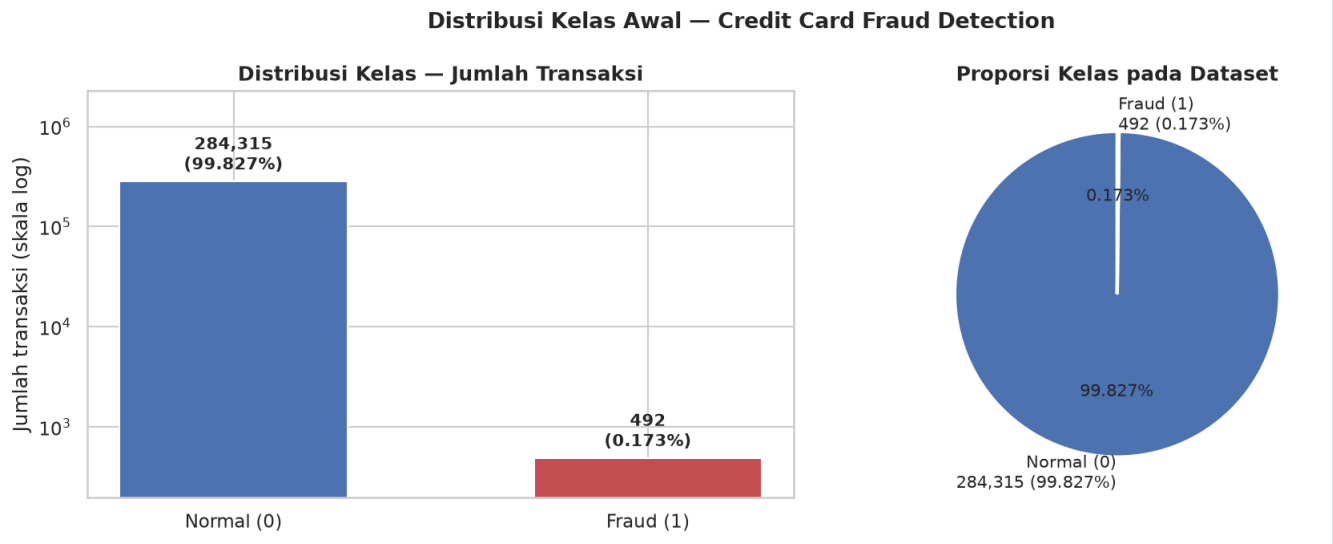
- **Precision** = TP / (TP + FP) — dari semua transaksi yang *diprediksi* fraud, berapa yang benar-benar fraud. Precision rendah berarti banyak nasabah yang salah diblokir.
- **Recall** = TP / (TP + FN) — dari semua fraud yang *sesungguhnya terjadi*, berapa yang berhasil ditangkap. Recall rendah berarti fraud lolos.
- **F1-score** — rata-rata harmonis precision dan recall; menyeimbangkan keduanya.
- **ROC-AUC** — kemampuan model memeringkat transaksi fraud lebih tinggi daripada transaksi normal, dihitung pada seluruh nilai threshold.
- **PR-AUC** (*Average Precision*) — area di bawah kurva precision-recall; lebih informatif daripada ROC-AUC pada data sangat timpang karena tidak melibatkan *true negative* yang jumlahnya mendominasi.

• **Confusion matrix** — menampilkan TN, FP, FN, dan TP secara eksplisit.

3. Hasil dan Analisis

3.1 Distribusi Kelas Awal

Distribusi kelas pada dataset asli sangat tidak seimbang: **284.315 transaksi normal (99,8273%)** berbanding **492 transaksi fraud (0,1727%)**. Ini berarti rata-rata hanya ada **1 transaksi fraud untuk setiap ±578 transaksi normal**.



Gambar 1. Distribusi kelas pada dataset awal. Batang kiri memakai skala log agar kelas fraud tetap terlihat; diagram kanan menunjukkan proporsi relatif kedua kelas.

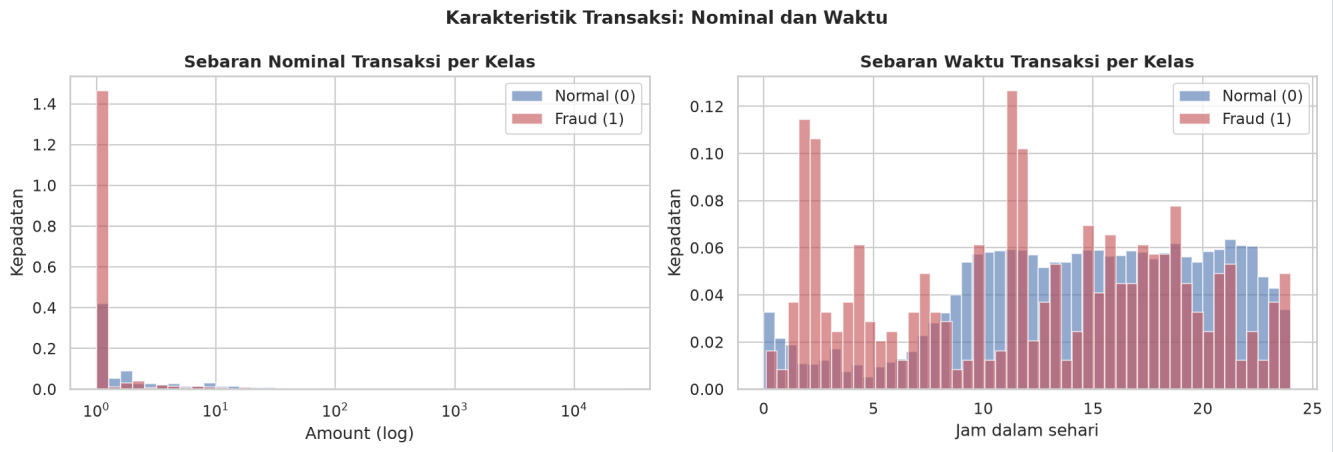
Dampak langsung dari komposisi ini: jika model memprediksi seluruh transaksi sebagai "Normal", maka model tersebut tetap benar pada 284.315 baris dan hanya salah pada 492 baris — akurasi 99,83% tanpa sedikit pun kemampuan mendeteksi fraud. Inilah alasan utama mengapa penanganan *class imbalance* menjadi inti persoalan, bukan sekadar penyempurnaan tambahan.

3.2 Karakteristik Transaksi Fraud

Eksplorasi awal menunjukkan pola yang menarik pada nominal transaksi: transaksi fraud memiliki **rata-rata Amount lebih tinggi** (122,21) daripada transaksi normal (88,29), tetapi **mediannya jauh lebih rendah** (9,25 berbanding 22,00). Artinya mayoritas transaksi fraud bernilai kecil (khas percobaan kartu / *card testing*), namun terdapat sebagian kecil transaksi bernilai sangat besar yang menarik rata-rata ke atas. Distribusi nominal transaksi pun saling tumpang tindih, sehingga `Amount` saja tidak cukup untuk memisahkan kedua kelas.

Tabel 4. Statistik nominal transaksi (`Amount`) per kelas

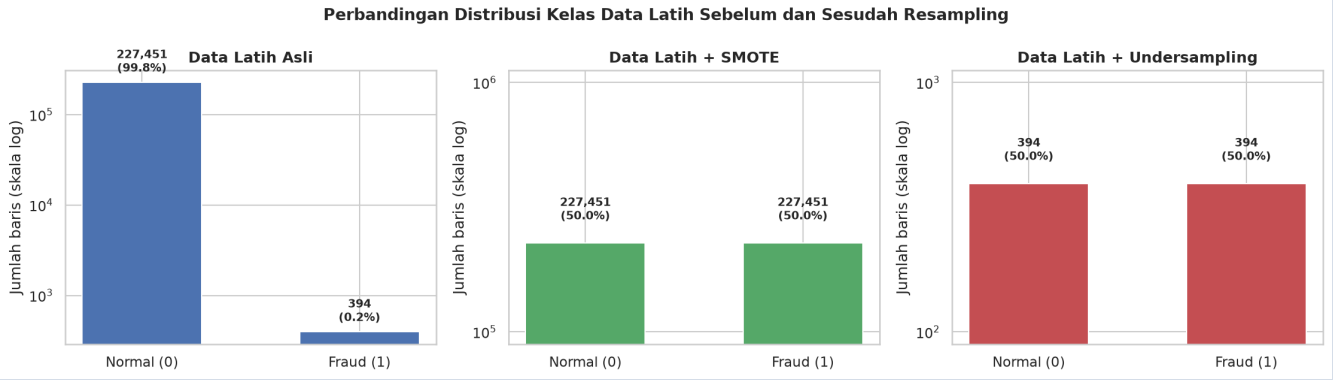
Kelas	Jumlah	Rata-rata	Median	Std. deviasi	Maksimum
Normal (0)	284.315	88,29	22,00	250,11	25.691,16
Fraud (1)	492	122,21	9,25	256,68	2.125,87



Gambar 2. Sebaran nominal transaksi (kiri, sumbu-x logaritmik) dan sebaran jam transaksi (kanan) untuk kedua kelas. Keduanya dinormalisasi sebagai kepadatan agar kelas fraud yang jumlahnya sedikit tetap dapat dibandingkan.

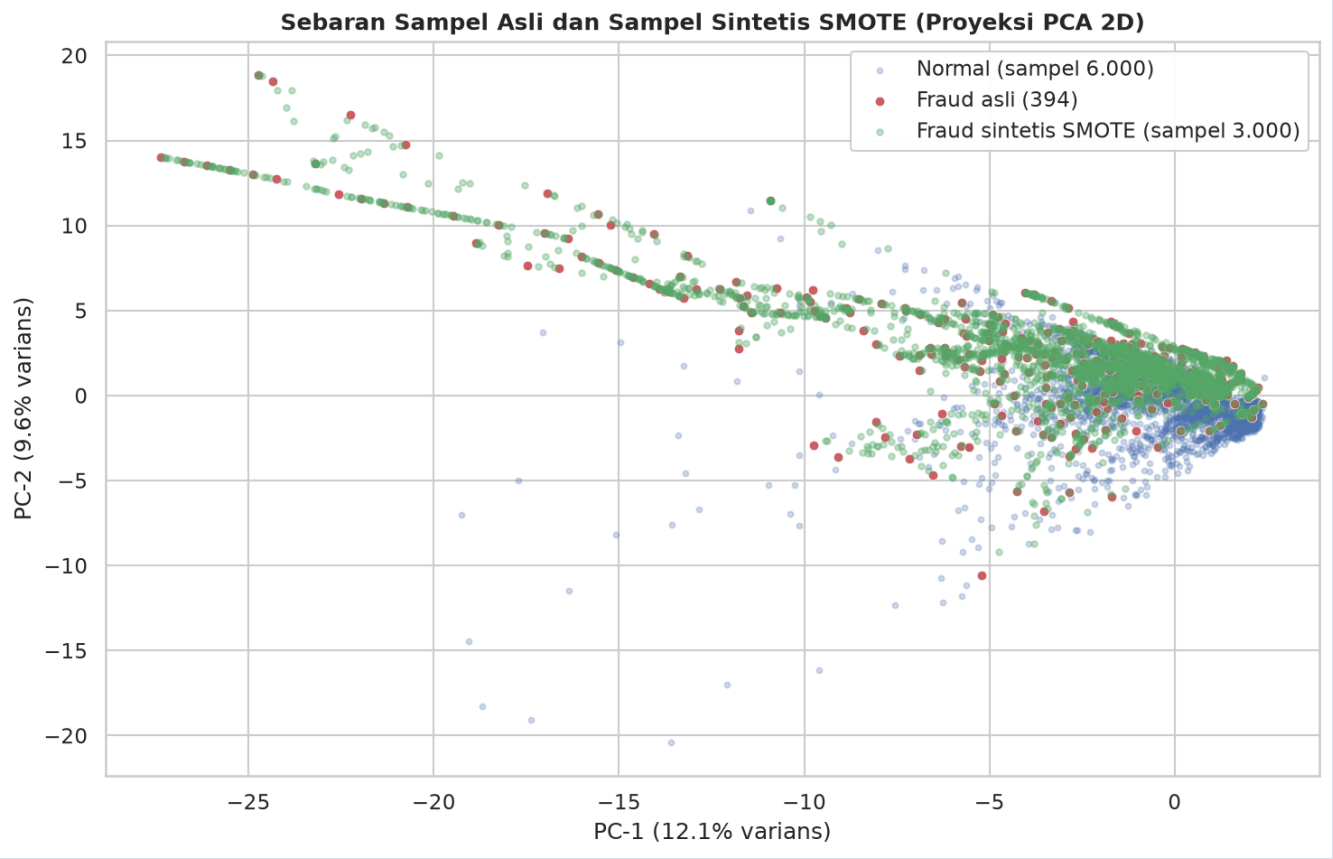
3.3 Hasil Penanganan Class Imbalance

Resampling hanya dikenakan pada data latih. SMOTE menyeimbangkan kelas dengan memperbanyak sampel fraud sintetis (394 → 227.451), sedangkan undersampling menyeimbangkan kelas dengan mengurangi sampel normal (227.451 → 394). Data uji tetap memiliki proporsi fraud 0,17% pada kedua skenario.



Gambar 3. Distribusi kelas data latih sebelum dan sesudah resampling (skala logaritmik). Kedua metode menghasilkan komposisi 50%:50%, tetapi dengan konsekuensi yang sangat berbeda terhadap jumlah informasi yang tersedia.

Perbedaan mendasar kedua metode terlihat dari sebaran sampelnya di ruang fitur (Gambar 4). Sampel sintetis SMOTE (hijau) menyebar **mengikuti pola transaksi fraud asli** (merah) dan berada di wilayah yang tumpang tindih dengan transaksi normal (biru) — konsisten dengan kenyataan bahwa fraud memang menyamar di antara transaksi normal. Karena itu, penambahan sampel sintetis tidak membuat persoalan menjadi mudah, tetapi memberi algoritma cukup contoh untuk mempelajari batas keputusan yang memisahkan kelas minoritas.



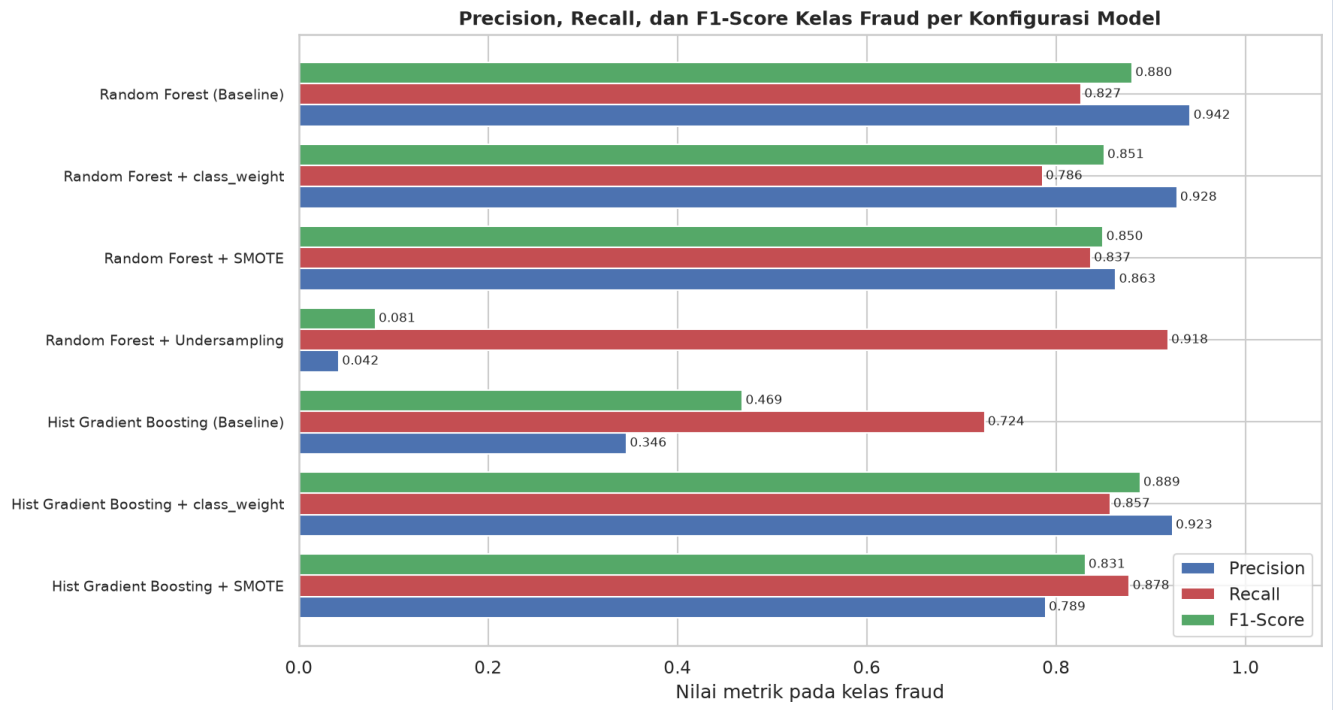
Gambar 4. Proyeksi PCA 2D data latih: transaksi normal (sampel 6.000), fraud asli (394), dan fraud sintetis hasil SMOTE (sampel 3.000). Sampel sintetis mengikuti sebaran fraud asli, bukan tersebar acak di seluruh ruang fitur.

3.4 Perbandingan Performa Model

Tabel 5 merangkum performa kedelapan konfigurasi pada data uji yang berdistribusi asli (98 transaksi fraud).

Tabel 5. Perbandingan performa seluruh konfigurasi model pada data uji (kelas fraud = positif). Baris hijau adalah model dengan F1-score tertinggi.

Model	Penanganan imbalance	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC	TN	FP	FN	TP
Random Forest (Baseline)	Tidak ada	0,9996	0,9419	0,8265	0,8804	0,9623	0,8731	56.859	5	17	81
Random Forest + `class_weight`	Pembobotan kelas	0,9995	0,9277	0,7857	0,8508	0,9613	0,8669	56.858	6	21	77
Random Forest + SMOTE	Oversampling (SMOTE)	0,9995	0,8632	0,8367	0,8497	0,9754	0,8749	56.851	13	16	82
Random Forest + Undersampling	Undersampling	0,9642	0,0424	0,9184	0,0810	0,9779	0,6927	54.830	2.034	8	90
Hist Gradient Boosting (Baseline)	Tidak ada	0,9972	0,3463	0,7245	0,4686	0,8539	0,4781	56.730	134	27	71
Hist Gradient Boosting + `class_weight`	**Pembobotan kelas**	**0,9996**	**0,9231**	**0,8571**	**0,8889**	**0,9828**	**0,8808**	**56.857**	**7**	**14**	**84**
Hist Gradient Boosting + SMOTE	Oversampling (SMOTE)	0,9994	0,7890	0,8776	0,8309	0,9769	0,8708	56.841	23	12	86
Dummy Classifier (pembanding)	*Tidak ada — selalu "Normal"*	*0,9983*	*0,0000*	*0,0000*	*0,0000*	*0,5000*	*0,0017*	*56.864*	*0*	*98*	*0*



Gambar 5. Precision, recall, dan F1-score kelas fraud untuk setiap konfigurasi ensemble. Konfigurasi undersampling menunjukkan pola ekstrem: recall sangat tinggi tetapi precision sangat rendah.

Pengaruh penanganan imbalance pada Random Forest

Random Forest dengan 200 pohon sudah relatif tangguh terhadap ketidakseimbangan kelas: baseline-nya mencapai **F1 0,8804** dengan recall 0,8265 dan precision 0,9419. Pada model ini, penanganan imbalance memberi perubahan yang **kecil dan tidak selalu searah**:

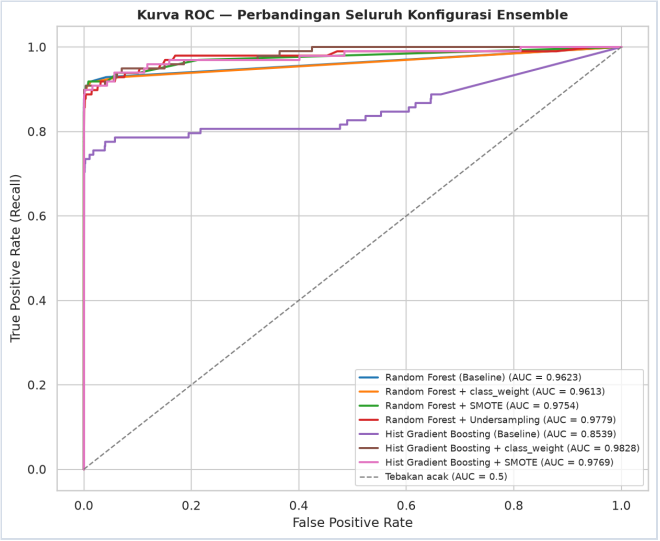
- **SMOTE** menaikkan recall **+1,02 poin persen** (0,8265 → 0,8367), menurunkan fraud yang lolos dari 17 menjadi 16, dan **menaikkan ROC-AUC dari 0,9623 ke 0,9754**. Namun precision turun dari 0,9419 ke 0,8632 (false alarm naik dari 5 ke 13), sehingga F1 justru turun tipis −3,07 poin persen.
- **Pembobotan kelas** justru *menurunkan* recall menjadi 0,7857 (−4,08 poin persen). Ini terjadi karena bobot kelas yang ekstrem menggeser letak batas keputusan sehingga lebih banyak transaksi berada di sekitar threshold 0,5 — menegaskan bahwa efek pembobotan kelas bergantung pada threshold keputusan yang dipakai.
- **Undersampling** menaikkan recall **+9,18 poin persen** menjadi 0,9184 (hanya 8 fraud lolos), tetapi **menghancurkan precision** menjadi 0,0424 — F1 jatuh −79,94 poin persen menjadi 0,0810. Model harus memblokir 2.124 transaksi untuk menangkap 90 fraud, artinya 2.034 transaksi sah menjadi korban *false alarm*.

Pengaruh penanganan imbalance pada Hist Gradient Boosting

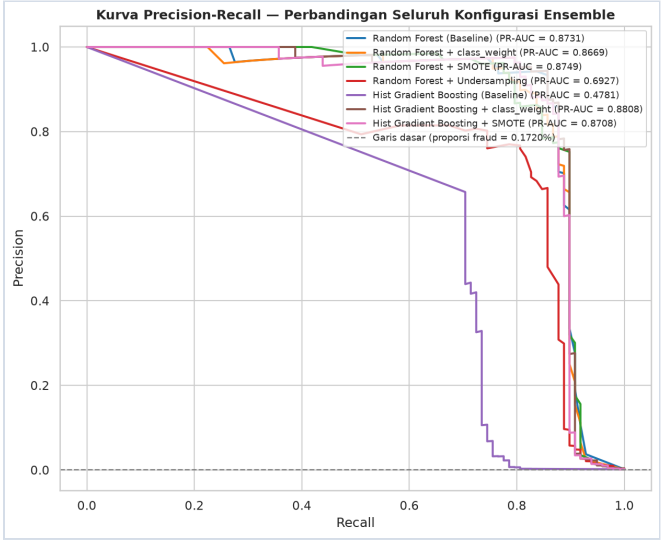
Berbeda jauh dari Random Forest, Hist Gradient Boosting **sangat bergantung** pada penanganan imbalance. Tanpa perlakuan apa pun, model ini hanya memperoleh **F1 0,4686** dengan precision 0,3463, ROC-AUC 0,8539, 27 fraud lolos, dan 134 false alarm — kinerja terburuk di antara seluruh model ensemble yang diuji. Setelah diberi penanganan imbalance, performanya melonjak tajam:

- **Pembobotan kelas**: recall **+13,27** dan F1 **+42,02 poin persen** (0,4686 → 0,8889), precision 0,3463 → 0,9231, ROC-AUC 0,8539 → 0,9828. Jumlah fraud yang lolos turun dari 27 menjadi 14, sementara false alarm turun drastis dari 134 menjadi 7.
- **SMOTE**: recall **+15,31** dan F1 **+36,23 poin persen**, dengan jumlah fraud lolos paling sedikit di antara model Gradient Boosting (12 transaksi). Harga yang dibayar adalah precision lebih rendah (0,7890) dan waktu latih 2,5× lebih lama karena data latih berlipat menjadi 454.902 baris.

3.5 Kurva ROC dan Precision-Recall



Gambar 6. Kurva ROC seluruh konfigurasi ensemble.

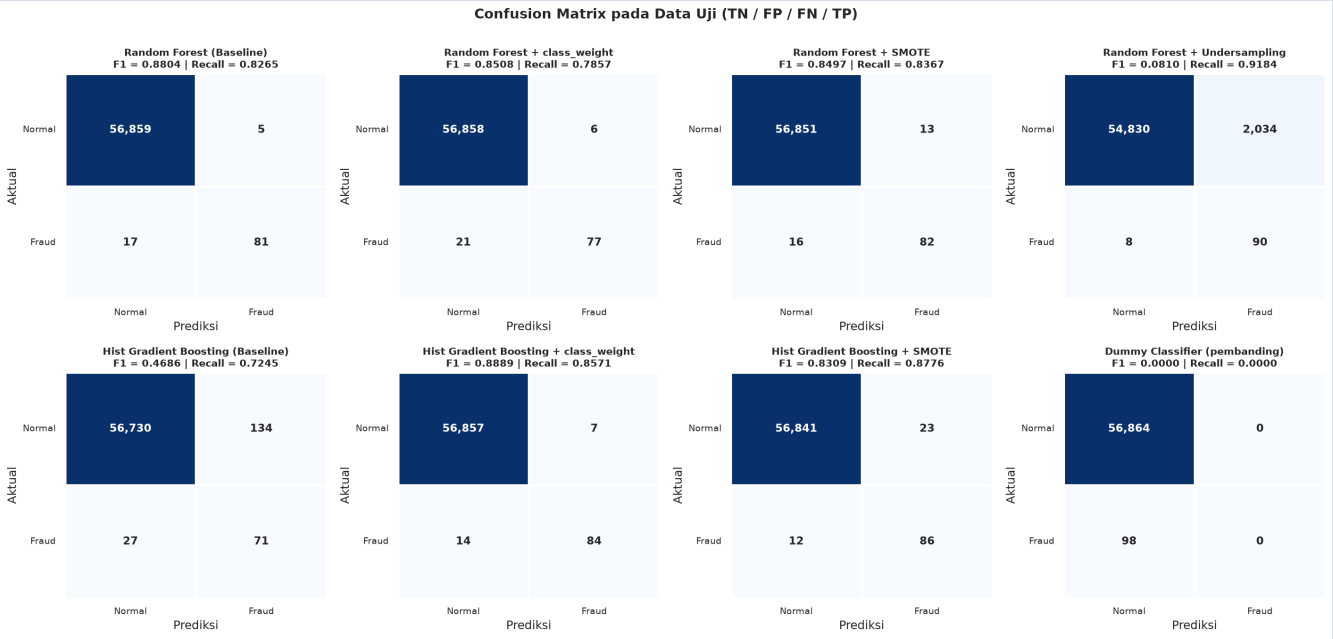


Gambar 7. Kurva Precision-Recall. Garis putus-putus menandai garis dasar proporsi fraud (0,1720%).

Kurva ROC menunjukkan bahwa hampir semua model yang ditangani memiliki ROC-AUC di atas 0,96 — **Hist Gradient Boosting + `class_weight` mencapai 0,9828**. Namun kurva ROC bersifat optimistis pada data timpang

karena *false positive rate* dihitung terhadap 56.864 transaksi normal, sehingga kesalahan pada kelas minoritas tampak "kecil". Kurva Precision-Recall (Gambar 7) memberikan gambaran yang lebih jujur: model terbaik mempertahankan precision di atas 0,85 hingga recall ±0,86 (PR-AUC 0,8808), sedangkan model undersampling jatuh cepat ke precision di bawah 0,1 (PR-AUC 0,6927) meskipun urutan peringkatnya (ROC-AUC 0,9779) terlihat tinggi. **Perbedaan ROC-AUC dan PR-AUC inilah bukti konkret mengapa pemilihan model pada data timpang tidak boleh hanya bersandar pada ROC-AUC.**

3.6 Confusion Matrix



Gambar 8. Confusion matrix seluruh konfigurasi pada 56.962 transaksi data uji (98 di antaranya fraud). Perhatikan sel Fraud–Normal (FN): jumlah fraud yang lolos deteksi.

Confusion matrix memperjelas pola yang tidak terlihat dari metrik agregat. Seluruh model yang ditangani imbalance berhasil menekan FN (fraud lolos) di bawah 17 kasus — terbaik adalah Random Forest dengan undersampling (8 FN) dan Hist Gradient Boosting dengan SMOTE (12 FN). Namun penekanan FN pada konfigurasi undersampling harus dibayar dengan ledakan FP menjadi 2.034 kasus, sedangkan **Hist Gradient Boosting + `class_weight`** menekan FN hingga 14 kasus **sekaligus** menahan FP pada 7 kasus saja — kombinasi terbaik antara daya tangkap dan ketepatan.

3.7 Mengapa Accuracy Saja Tidak Cukup?

Bukti paling langsung datang dari `DummyClassifier` yang selalu memprediksi "Normal" untuk setiap transaksi:

Tabel 6. Perbandingan model "pintar" dan model "malas" pada data uji

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	Fraud tertangkap	Fraud lolos
Dummy Classifier (selalu "Normal")	99,8280%	0,0000	0,0000	0,0000	0,5000	0 / 98	98
Random Forest (Baseline)	99,9614%	0,9419	0,8265	0,8804	0,9623	81 / 98	17
Hist Gradient Boosting + `class_weight`	**99,9631%**	**0,9231**	**0,8571**	**0,8889**	**0,9828**	**84 / 98**	**14**

Model *dummy* memperoleh accuracy **99,8280%** — hanya **0,1352 poin persen** di bawah model terbaik — padahal model tersebut **tidak mendeteksi satu pun transaksi fraud**. Recall, precision, dan F1-score-nya nol, dan ROC-AUC-nya tepat 0,5 (setara tebakan acak, karena semua transaksi diberi peluang yang sama).

Penyebabnya adalah aritmetika *accuracy* itu sendiri pada data timpang:

- Accuracy dihitung dari **seluruh 56.962 transaksi**, di mana **56.864 di antaranya (99,83%)** adalah transaksi normal yang sangat mudah dikenali. Kontribusi kelas mayoritas menenggelamkan kesalahan pada kelas minoritas.
- Selisih accuracy antara model tanpa kemampuan deteksi dan model terbaik adalah **0,1352 poin persen**, sementara selisih **recall-nya 85,71 poin persen** (0,0000 → 0,8571). Artinya, accuracy praktis tidak sensitif terhadap kemampuan mendeteksi fraud.
- Pada **threshold keputusan 0,5** yang menjadi bawaan library, model dapat terlihat "hampir sempurna" hanya dengan berpihak ke kelas mayoritas — padahal peluang fraud yang diprediksi tetap dapat mengurutkan transaksi dengan baik (ROC-AUC 0,98). Metrik berbasis ranking dan metrik kelas minoritas menangkap perbedaan ini, accuracy tidak.

Oleh karena itu, penilaian model pada kasus ini menggunakan **precision, recall, F1-score, ROC-AUC, PR-AUC, dan confusion matrix pada kelas fraud**. Dari sisi bisnis, kesalahan tipe **FN (fraud lolos)** menimbulkan kerugian finansial langsung yang besarnya berlipat, sementara **FP (transaksi normal diblokir)** hanya menimbulkan biaya verifikasi dan ketidaknyamanan nasabah — sehingga **recall** menjadi metrik paling kritis, dengan batas toleransi tertentu pada precision.

3.8 Model Terbaik dan Trade-off-nya

Hist Gradient Boosting + pembobotan kelas adalah model terbaik secara keseluruhan: F1-score 0,8889 (tertinggi), precision 0,9231, recall 0,8571, ROC-AUC 0,9828, PR-AUC 0,8808, dengan 14 fraud lolos dan hanya 7 *false alarm*. Model ini menangkap **84 dari 98 transaksi fraud (85,71%)** dan hanya salah menandai **0,0123%** transaksi normal.

Jika prioritas organisasi adalah memaksimalkan penangkapan fraud tanpa memedulikan biaya penanganan *false alarm*, maka **Random Forest + undersampling** menangkap 90 dari 98 fraud (recall 0,9184) — tetapi harus memblokir 2.124 transaksi untuk mendapatkan 90 fraud yang benar, sehingga ketepatan keputusannya rendah (precision 0,0424, F1 0,0810). Perbandingan ini menunjukkan bahwa **tidak ada konfigurasi yang unggul di semua metrik**; pemilihan model harus mengikuti prioritas bisnis, dan hasil ini menegaskan kembali bahwa metrik tunggal apa pun (termasuk accuracy) tidak dapat merangkum performa model pada data tidak seimbang.

4. Pembahasan

1. **Efek penanganan imbalance bersifat spesifik terhadap algoritma, bukan universal.** Random Forest 200 pohon sudah mencapai F1 0,8804 tanpa perlakuan apa pun, sehingga penambahan SMOTE hanya menggeser titik operasi (recall naik, precision turun) dan bahkan menurunkan F1 secara tipis. Sebaliknya, pada Hist Gradient Boosting efeknya besar dan konsisten: F1 melonjak dari 0,4686 ke 0,8889. Kesimpulan praktisnya: penanganan imbalance tetap diperlukan, tetapi harus divalidasi per model — bukan diterapkan sebagai resep seragam.
2. **Pembobotan kelas memberi rasio manfaat terhadap biaya terbaik.** Pendekatan ini menghasilkan model dengan F1-score tertinggi tanpa menambah satu baris data pun (waktu latih ±6 detik, dibanding ±16 detik untuk SMOTE), sehingga paling efisien untuk iterasi eksperimen dan paling mudah dipertahankan pada pipeline produksi.
3. **SMOTE lebih efektif menjaga informasi kelas mayoritas** dibanding undersampling: SMOTE mempertahankan seluruh 227.451 transaksi normal dan menambahkan sampel sintetis, sehingga model masih belajar variasi transaksi normal yang luas (FP terkendali pada 13–23 kasus). Undersampling membuang 99,8% data normal dan akibatnya model kehilangan kemampuan mengenali keragaman transaksi normal — precision runtuh ke 0,0424.
4. **Undersampling bukan pilihan yang tepat untuk dataset ini** meskipun memberikan recall tertinggi. Pelatihannya memang sangat cepat (±0,3 detik, dibanding ±77–126 detik Random Forest pada data penuh), tetapi biaya operasionalnya tidak dapat diterima: 2.034 nasabah akan menerima pemblokiran atau verifikasi yang tidak perlu.
5. **Semua model terbaik masih menyisakan FN.** Bahkan konfigurasi terbaik melewatkan 12–14 dari 98 transaksi fraud. Karena itu penyetelan *threshold* keputusan dan analisis biaya eksplisit (biaya FN vs FP) merupakan langkah lanjutan yang lebih berdaya guna daripada sekadar mengganti algoritma.

5. Kesimpulan

1. Dataset memiliki **class imbalance ekstrem**: 492 fraud dari 284.807 transaksi (0,1727%), dengan rasio ±578 : 1. Pembagian 80:20 dengan `stratify=y` menghasilkan data latih 227.845 baris (394 fraud) dan data uji 56.962 baris (98 fraud) dengan proporsi kelas yang terjaga.
2. Resampling **hanya diterapkan pada data latih**: SMOTE menyeimbangkan kelas menjadi 454.902 baris (50%:50%) dan random undersampling menjadi 788 baris (50%:50%), sedangkan data uji tetap memakai distribusi asli 0,17% fraud agar evaluasi realistis.
3. Model terbaik adalah **Hist Gradient Boosting + `class_weight="balanced"`** dengan **F1-score 0,8889**, precision 0,9231, recall 0,8571, ROC-AUC 0,9828, dan PR-AUC 0,8808 — menangkap 84 dari 98 fraud dengan hanya 7 false alarm. Trade-off terbaik ini dicapai **tanpa resampling sama sekali**.
4. Penanganan imbalance menaikkan performa Hist Gradient Boosting secara drastis (F1 **+42,02 poin persen** untuk pembobotan kelas; **+36,23 poin persen** untuk SMOTE), tetapi pengaruhnya kecil bahkan sedikit negatif pada Random Forest (F1 −3,07 poin persen untuk SMOTE), kecuali pada konfigurasi undersampling yang menaikkan recall +9,18 poin persen dengan mengorbankan precision secara ekstrem (F1 −79,94 poin persen).
5. **Accuracy tidak layak menjadi metrik utama** untuk dataset ini: model yang selalu memprediksi "Normal" memperoleh accuracy 99,8280% — hanya 0,1352 poin persen di bawah model terbaik — padahal recall-nya 0 dan seluruh 98 transaksi fraud lolos. Evaluasi harus bersandar pada **precision, recall, F1-score, ROC-AUC, PR-AUC, dan confusion matrix kelas fraud**, dengan **recall** sebagai prioritas karena biaya FN jauh melampaui biaya FP.

6. Rekomendasi

- **Pemilihan model**: gunakan F1-score atau recall kelas fraud sebagai kriteria utama, dan laporkan PR-AUC bersama ROC-AUC karena PR-AUC tidak terdistorsi oleh banyaknya *true negative*.
- **Penyetelan threshold**: jangan berhenti pada threshold bawaan 0,5 — pilih titik operasi pada kurva precision-recall sesuai toleransi tim antifraud, dan validasi dengan data berurutan waktu (*time-based split*) untuk menguji kestabilan.
- **Cost-sensitive learning**: susun matriks biaya eksplisit (biaya investigasi FP vs kerugian FN) lalu kalibrasi decision threshold untuk meminimalkan total biaya, bukan memaksimalkan metrik statistik.
- **Eksplorasi lanjutan**: bandingkan dengan ADASYN, SMOTE-ENN, atau `BalancedRandomForestClassifier`; tambahkan *hyperparameter tuning* (misalnya `max_depth`, `min_samples_leaf`) yang dioptimalkan untuk F1 kelas fraud, bukan accuracy.
- **Keterbatasan yang perlu dicatat**: (a) fitur `V1` – `V28` adalah hasil PCA sehingga tidak dapat diinterpretasikan secara bisnis; (b) data hanya mencakup dua hari transaksi sehingga pola musiman jangka panjang belum terwakili; (c) terdapat 1.081 baris duplikat pada dataset asli yang berpotensi membuat evaluasi terlalu optimistis bila baris identik muncul pada data latih dan data uji sekaligus.

Lampiran — Berkas Pendukung

Berkas	Keterangan
`5054251003_FirdausMangkona_Tugas_Ensemble.ipynb`	Notebook utama: pemuatan data, praproses, resampling, pelatihan 8 konfigurasi model, evaluasi, dan seluruh visualisasi (sudah dieksekusi lengkap dengan output)
`data/creditcard.csv`	Dataset mentah (284.807 transaksi × 31 kolom)
`figures/01-08_*.png`	Delapan visualisasi yang dihasilkan notebook dan dipakai pada laporan ini
`results/metrics_comparison.csv`	Tabel lengkap seluruh metrik, confusion matrix, dan waktu latih tiap konfigurasi
`results/experiment_summary.json`	Ringkasan konfigurasi eksperimen (ukuran data, hasil resampling, metrik, model terbaik)

Lingkungan eksekusi. Python 3.14.7 dengan kernel `.venv` proyek, pandas 3.0.6, numpy 2.5.4, scikit-learn 1.9.1, imbalanced-learn 0.14.2, matplotlib 3.11.2, seaborn 0.13.2. Seluruh eksperimen menggunakan `random_state=42` sehingga hasil dapat direproduksi.